

# MCP サーバー開発ハンズオン

ハンズオン資料

## MCP サーバー開発ハンズオン

- 1: [MCP とは？](#)
- 2: [はじめての MCP サーバー](#)
- 3: [自分だけの MCP サーバーを作成する](#)
- 4: [MCP サーバーを Streamable HTTP に対応させる](#)
- 5: [セッション管理機能をもつ MCP サーバーを開発する](#)

このハンズオンでは、MCP (Model Context Protocol) サーバーを設計・実装し、実際に動作させるまでの流れを体験します。

今回開発する MCP サーバーのコードは、以下の GitHub リポジトリに公開されています。

- [ry0y4n/sample-mcp-server](#)

また、本ハンズオンを GitHub Codespaces 上で実行するためのサンプルリポジトリも用意しています。

- [sample-mcp-server-for-codespaces](#)

## ハンズオンの目次

# 1 - MCP とは？

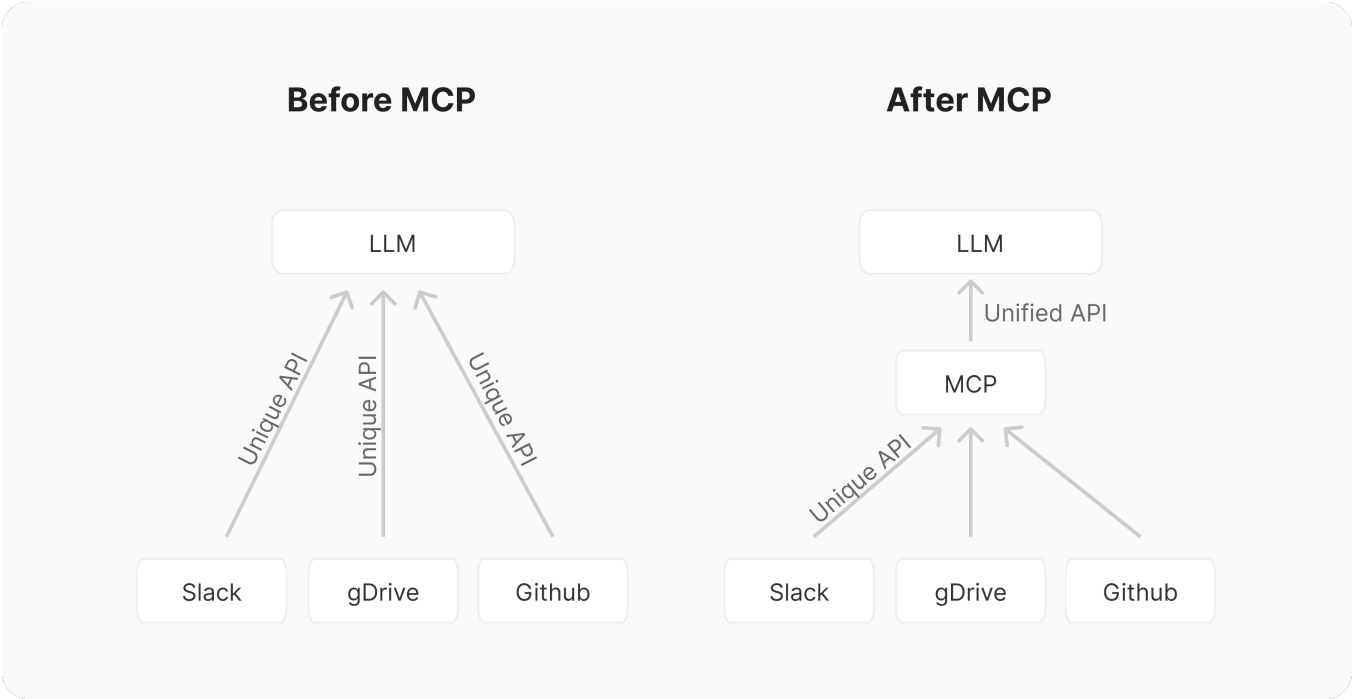
## MCP とは？

**MCP (Model Context Protocol)** とは、LLM (大規模言語モデル) が外部のツールやデータに安全かつ汎用的にアクセスできるようにするための オープンな標準プロトコル です。Anthropic を中心に策定が進められており、“AI 業界の USB-C” とも呼ばれるほど、**LLM** と外部世界をつなぐ共通インターフェース として期待されています。

🔗 公式サイト : <https://modelcontextprotocol.io>

## 背景：なぜ MCP が必要なのか？

従来、LLM に外部データやアクションを渡すには、モデルごと・サービスごとに個別に実装する必要があり、開発・運用コストが高騰していました。MCP はこの課題を解決し、「**M × N**」個の接続が必要だった世界を「**M + N**」個で済ませられる世界へと変えます。



[Philschmid - Model Context Protocol \(MCP\) an overview](#) より引用

## MCP の基本構成

MCP サーバーは大きく分けて以下 3 つの機能で構成されています。

| 名前 | 説明 | 主な役割 |
|----|----|------|
|----|----|------|

| 名前        | 説明                           | 主な役割          |
|-----------|------------------------------|---------------|
| Tools     | LLM が外部の機能呼び出すための「ツール」       | モデルによるアクション実行 |
| Resources | LLM が参照する静的な情報リソース           | アプリケーション的な制御  |
| Prompts   | ユーザーが選択する事前定義されたプロンプト テンプレート | ユーザー側の制御      |

たとえば、天気予報 API を「Tool」として呼び出したり、クラウド上のドキュメントを「Resource」として参照したりすることで、より実用的な AI アプリケーションを構築できます。

2025 年 5 月 20 日時点では、VS Code では「Tool」機能のみがサポートされています。  
 [Supported MCP capabilities](#)

## 通信方式 (Transport)

MCP は以下のような通信方式をサポートしています：

- stdio：標準入出力（CLI ベースの開発に便利）
- SSE：Server-Sent Events（ストリーミング通信、今は非推奨気味）
- Streamable HTTP：高可用性・ステートレスな通信が可能な新方式（2025 年 4 月以降の推奨手段）

GitHub Copilot や Claude Desktop などのクライアントが、これらの方式を使って MCP サーバーと通信します。

## 対応クライアントと活用例

MCP をサポートするクライアントの各機能への対応状況は、以下の公式サイトで確認できます：

 [MCP クライアントの機能対応状況](#)

## Feature support matrix

| Client             | Resources | Prompts | Tools | Discovery | Sampling | Roots |                                                            |
|--------------------|-----------|---------|-------|-----------|----------|-------|------------------------------------------------------------|
| 5ire               | ✗         | ✗       | ✓     | ?         | ✗        | ✗     | Supports toc                                               |
| AgentAI            | ✗         | ✗       | ✓     | ?         | ✗        | ✗     | Agent Librar<br>with tools su                              |
| AgenticFlow        | ✓         | ✓       | ✓     | ✓         | ✗        | ✗     | Supports toc<br>resources fo<br>agents and r<br>workflows. |
| Amazon Q CLI       | ✗         | ✓       | ✓     | ?         | ✗        | ✗     | Supports pro                                               |
| Apify MCP Tester   | ✗         | ✗       | ✓     | ✓         | ✗        | ✗     | Supports toc                                               |
| BeeAI Framework    | ✗         | ✗       | ✓     | ✗         | ✗        | ✗     | Supports toc<br>workflows.                                 |
| BoltAI             | ✗         | ✗       | ✓     | ?         | ✗        | ✗     | Supports toc                                               |
| Claude.ai          | ✓         | ✓       | ✓     | ✗         | ✗        | ✗     | Supports toc<br>resources fo<br>servers.                   |
| Claude Code        | ✗         | ✓       | ✓     | ✗         | ✗        | ✗     | Supports pro                                               |
| Claude Desktop App | ✓         | ✓       | ✓     | ✗         | ✗        | ✗     | Supports toc<br>resources fo<br>MCP servers                |
| Cline              | ✓         | ✗       | ✓     | ✓         | ✗        | ✗     | Supports toc                                               |
| Continue           | ✓         | ✓       | ✓     | ?         | ✗        | ✗     | Supports toc                                               |

このページでは、各クライアントがサポートする機能（Resources, Prompts, Tools など）や、特定のユースケースにおける活用例が詳しく説明されています。

## MCP は何がすごいのか？

- ✓ **LLM** とツール/データの接続標準化：さまざまな LLM とツール/データを簡単に接続できる
- ✓ 開発しやすい：TypeScript や Python などの公式 SDK が充実
- ✓ オープン性と拡張性：だれでも MCP サーバーを作って公開できる
- ✓ **Azure** と連携：Azure Functions や API Management と統合して企業利用も可能

## まとめ

MCP は、LLM が「外の世界」と自然に対話するための標準規格です。

自分だけの MCP サーバーを構築すれば、LLM に独自の機能を呼び出させたり、好きなデータを扱わせたりすることができます。

■ 次のページでは、実際に MCP サーバーを作成しながら仕組みを体験していきます！

## 2 - はじめての MCP サーバー

### はじめての MCP サーバーを使ってみよう

このページでは、MCP クライアントのひとつである **GitHub Copilot** のエージェントモードを使って、公式で公開されている MCP サーバーのひとつ **Time MCP** を利用してみます。

 Time MCP は、現在時刻を返してくれるシンプルな MCP サーバーです。

リポジトリ: [modelcontextprotocol/servers/src/time](https://github.com/modelcontextprotocol/servers/tree/main/src/time)

### このページのゴール

- GitHub Copilot のエージェントモードを起動する
- Time MCP サーバーを選択して使用する
- MCP の基本的な動作を体験する

### 1. Time MCP をインストールする

Time MCP のリポジトリ ([こちら](#)) にアクセスし、以下の手順で MCP サーバーをインストールします。

1. Time MCP のリポジトリにアクセスする
2. 「Configure for VS Code」セクションを探し、VS Code 用のインストール ボタンをクリックする
  - uv と docker を使ったインストール方法が表示されますが、ここでは docker を選択します（お好きな方を選んでください）
3. VS Code が開き、モーダルが表示されるので「**Install Server**」を選択します

### Configure for VS Code

For quick installation, use one of the one-click install buttons below...

**VS Code** **UV** **VS Code Insiders** **UV**

**VS Code** **Docker** **VS Code Insiders** **Docker**

For manual installation, add the following JSON block to your User Settings (JSON) file in VS Code. You can do this by pressing **Ctrl + Shift + P** and typing **Preferences: Open User Settings (JSON)**.

Optionally, you can add it to a file called `.vscode/mcp.json` in your workspace. This will allow you to share the configuration with others.

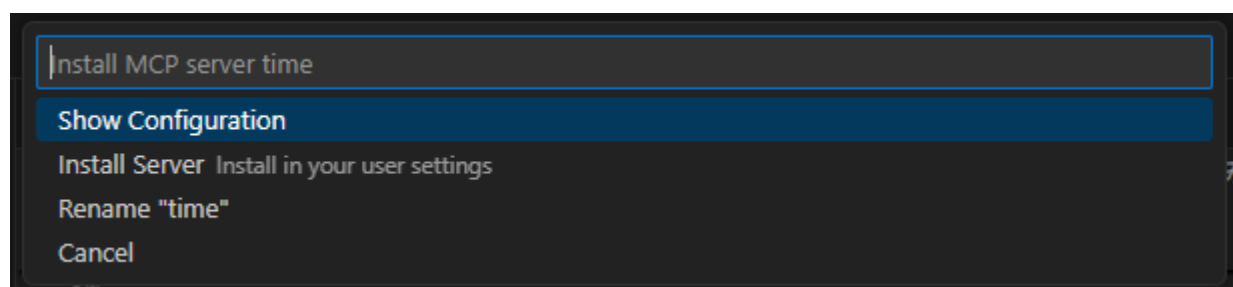
Note that the `mcp` key is needed when using the `mcp.json` file.

▼ Using uvx

```
{
  "mcp": {
    "servers": {
      "time": {
        "command": "uvx",
        "args": ["mcp-server-time"]
      }
    }
  }
}
```

▼ Using Docker

```
{
  "mcp": {
    "servers": {
      "time": {
        "command": "docker",
        "args": ["run", "-i", "--rm", "mcp/time"]
      }
    }
  }
}
```



インストールが完了すると、VS Code の設定ファイル（`settings.json`）に以下のような設定が追加されます：

```
{
  "mcp": {
    "servers": {
      // ...Existing MCP servers...

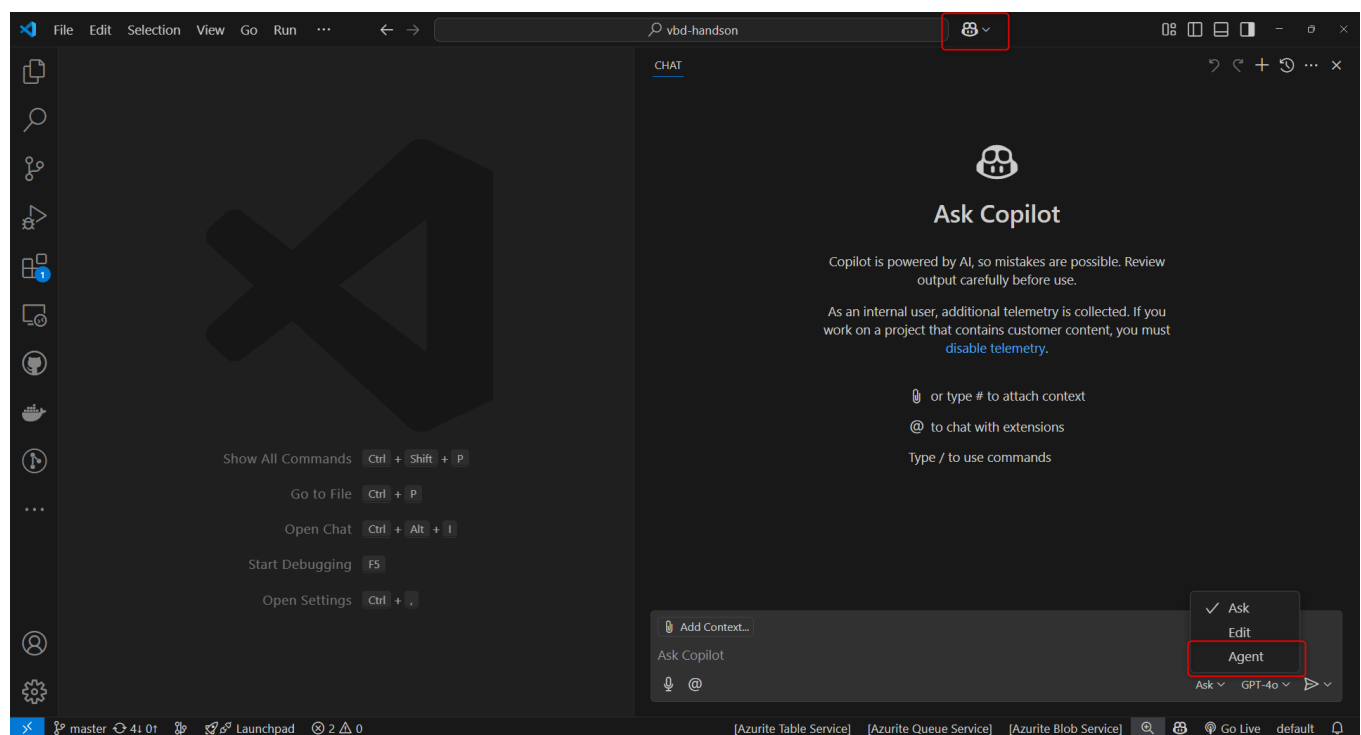
      "time": {
        "command": "docker",
        "args": ["run", "-i", "--rm", "mcp/time"]
      }
    }
  }
}
```

VS Code では上記のように **settings.json** に MCP サーバーの設定を追加することで、サーバーを認識します。

## 2. MCP ツールを試してみる

VS Code (v1.99 以降) で GitHub Copilot を開き、以下の手順でエージェントモードを起動します。

1. VS Code 上部の **GitHub Copilot** アイコン をクリックして、Copilot チャット画面を開く
2. チャット欄からモードを「**Agent**」モードを選択する

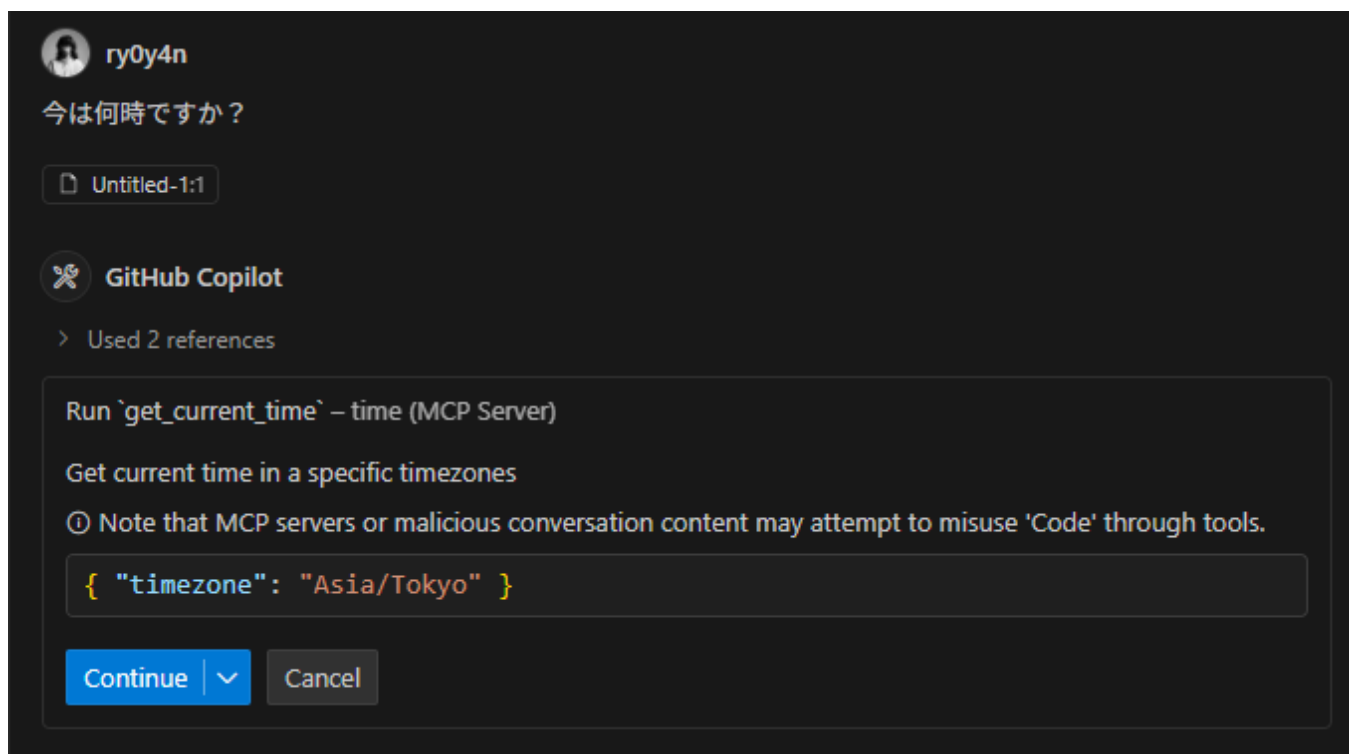


3. チャット入力欄に「今は何時ですか?」と入力し、Enter キーを押す
4. Copilot が「**Time MCP**」を選択し、現在時刻を返してくれます



チャット画面で以下のように入力します：

今は何時ですか？



すると上記のように `get_current_time` というツールを使ってよいかを聞いてきますので、「**Continue**」を選択して実行させます。

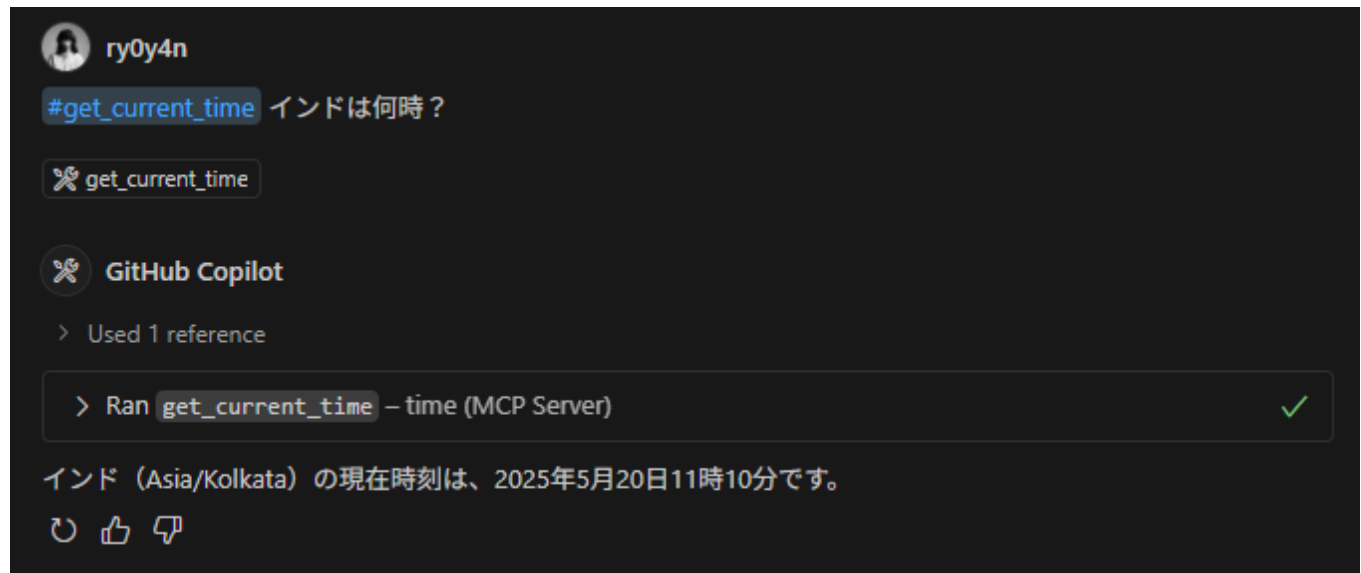


これにより、GitHub Copilot が Time MCP サーバーを呼び出し、現在時刻を返します。

## 指定ツールを直接呼び出す方法

通常だとチャットの内容に応じて GitHub Copilot が適切なツールを選択してくれますが、指定したツールを直接呼び出すこともできます。例えば、以下のように「**get\_current\_time**」を直接呼び出すことができます：

```
#get_current_time
```



すると、上記のように `get_current_time` を直接呼び出すことができ、Time MCP サーバーから現在時刻を取得することができます。

次のページから、MCP サーバーを自分で作成してみましょう。

## 3 - 自分だけの MCP サーバーを作成する

### はじめての MCP サーバーを作ってみよう！

ここでは、自分だけの簡単な MCP サーバーを作成し、GitHub Copilot などのクライアントと連携できるようになるまでの手順を解説します。

最終的には、ローカルにスニペットを保存&取得できる **MCP ツール** を実装します！

#### このページのゴール

- 自分だけの MCP サーバーを作成する
- GitHub Copilot と連携して MCP サーバーを利用する
- MCP サーバーの基本的な構築方法を学ぶ

## 1. プロジェクトを準備する

まずは作業用のディレクトリを作成して、必要なファイルと依存関係を準備します。

```
# ディレクトリ作成
mkdir sample-mcp-server
cd sample-mcp-server

# ファイル作成
touch stdio_server.ts # PowerShell: New-Item stdio_server.ts

# VS Code を開く
code .
```

次に、npm プロジェクトを初期化し、MCP サーバーの公式 SDK をインストールします。

```
npm init -y
npm install @modelcontextprotocol/sdk
npm install --save-dev @types/node
```

## 2. 最小構成の MCP サーバーを作る

まずは「2つの数を足し算するツール（add）」だけを持った、最小構成の MCP サーバーを作ってみましょう。

以下のコードを `stdio_server.ts` に貼り付けます。

```
import {
  McpServer,
  ResourceTemplate,
} from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

// MCP サーバーを作成
const server = new McpServer({
  name: "Demo",
  version: "1.0.0",
});

// add ツールを追加
server.tool(
  "add", // ツール名
  "Add two numbers", // ツールの説明
  { a: z.number(), b: z.number() }, // ツールの引数
  async ({ a, b }) => ({
    content: [{ type: "text", text: String(a + b) }],
  })
);

// 標準入出力で接続
const transport = new StdioServerTransport();
await server.connect(transport);
```

💡 MCP では、ツール（`tool(...)`）を通じて LLM が外部機能呼び出せるようになります。ここでは「`a + b` を計算する機能」を提供しています。

## 3. `settings.json` で VS Code に認識させる

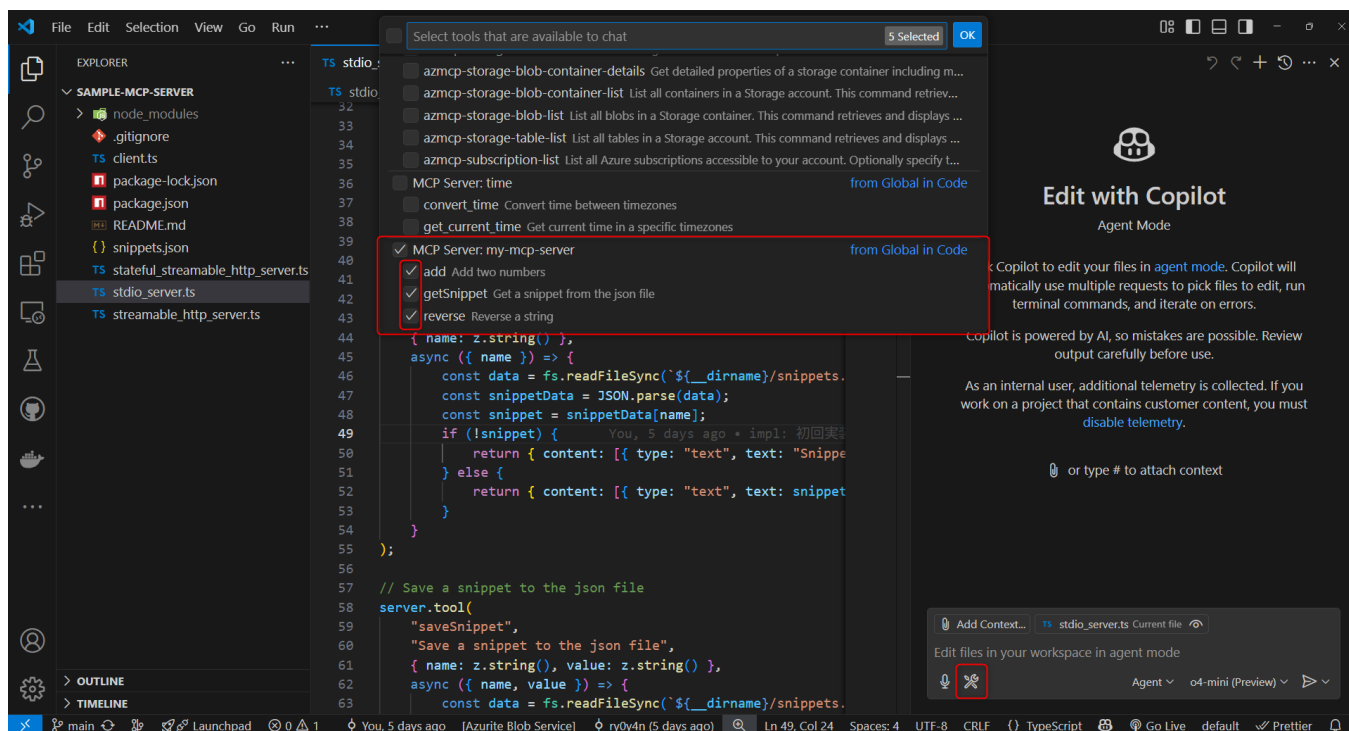
この MCP サーバーを GitHub Copilot で利用するには、VS Code の設定ファイルにサーバーの起動コマンドを記述します。

.vscode/settings.json に以下のように記載してください（パスは各自の環境に合わせて変更してください）。

```
{
  "mcp": {
    "servers": {
      // ...Existing Servers...
      "my-mcp-server": {
        "command": "node",
        "args": ["<stdio_server.ts までの絶対パス>/stdio_server.ts"]
      }
    }
  }
}
```

## 4. Copilot エージェント モードで使ってみよう

エージェント モードで VS Code を起動し、MCP サーバー一覧から my-mcp-server を選択します。



チャット画面で以下のように入力して、MCP サーバーが正しく動いているか確認してみましょう。

```
123 足す 456 は？
```



もし上手くツールが認識されない場合は、`#add` と入力して、明示的にツールを呼び出してみてください。

Copilot から「**579**」と返ってくれば成功です！

## 5. 🧠 スニペット保存ツールを追加しよう

次は、少しレベルアップしたツールを作ってみましょう。

ローカルにスニペットを保存・取得するツールです。

まずは JSON ファイルを作っておきます。

```
touch snippets.json # PowerShell: New-Item snippets.json
```



中身は以下のように空の JSON オブジェクトを記載しておきます。

```
{}
```



そして、`stdio_server.ts` の `add` ツールの下に、以下のコードを追加します。

```
import fs from "fs";

// スニペットを取得するツール
server.tool(
  "getSnippet",
  "Get a snippet from the json file",
  { name: z.string() },
  async ({ name }) => {
    const data = fs.readFileSync(`${__dirname}/snippets.json`, "utf8");
    const snippetData = JSON.parse(data);
    const snippet = snippetData[name];
    if (!snippet) {
      return { content: [{ type: "text", text: "Snippet not found" }] };
    } else {
```



```
        return { content: [{ type: "text", text: snippet }] };
    }
}
);

// スニペットを保存するツール
server.tool(
    "saveSnippet",
    "Save a snippet to the json file",
    { name: z.string(), value: z.string() },
    async ({ name, value }) => {
        const data = fs.readFileSync(`${__dirname}/snippets.json`, "utf8");
        const snippetData = JSON.parse(data);
        snippetData[name] = value;
        fs.writeFileSync(
            `${__dirname}/snippets.json`,
            JSON.stringify(snippetData, null, 2)
        );
        return { content: [{ type: "text", text: `Snippet ${name} saved` }] };
    }
);
```

これで、以下のようにしてスニペットの保存・取得ができます：

「hello」という名前で `console.log("Hello World!");` というスニペットを保存して



 ry0y4n

「hello」という名前で `console.log("Hello World!");` というスニペットを保存して

 GitHub Copilot

> Used 1 reference

▼ Ran `saveSnippet` – my-mcp-server (MCP Server) 

Input

```
{
  "name": "hello",
  "value": "console.log(\"Hello World!\");"
}
```

Output

```
Snippet hello saved
```

スニペット「hello」を保存しました。内容は `console.log("Hello World!");` です。

「hello」のスニペットを見せて



 ry0y4n

「hello」のスニペットを見せて

 GitHub Copilot

> Used 1 reference

▼ Ran `getSnippet` – my-mcp-server (MCP Server) ✓

**Input**

```
{
  "name": "hello"
}
```

**Output**

```
console.log("Hello World!");
```

スニペット「hello」の内容は以下の通りです：

```
console.log("Hello World!");
```

## おわりに

このページでは、MCP サーバーの基本構築とローカルでのツール提供の方法を学びました。stdio ベースの MCP サーバーはローカル開発には最適ですが、他の端末やクラウド上のクライアントから利用することはできません。

本格的に他のユーザーやツールと連携したい場合は、標準入出力ではなく **HTTP** 経由で通信するサーバー（＝リモート **MCP** サーバー）を構築する必要があります。

## ✓ 次のステップ

次の章では、MCP サーバーを Streamable HTTP 対応に書き換え、外部からアクセス可能な形にアップデートしていきます。

これは MCP サーバーを Azure Functions や App Service、Azure Container Apps などにデプロイして利用するための重要なステップです。

 「リモートからも使える自作 MCP サーバー」へ、いよいよ本格始動です！👉

stdio\_server.ts を Streamable HTTP 用書き換えて、クラウド対応の第一歩を踏み出



しまししょう！

# 4 - MCP サーバーを Streamable HTTP に対応させる

## MCP サーバーを Streamable HTTP に対応させる

このページでは、MCP サーバーを Streamable HTTP に対応させ、リモート環境でのホスティングが可能な形にアップデートします。

### このページのゴール

- MCP サーバーを Streamable HTTP に対応させる
- リモート環境での MCP サーバーのホスティングを学ぶ
- Streamable HTTP の基本的な実装方法を理解する

### MCP のトランスポート方式の変遷

MCP (Model Context Protocol) では、クライアントとサーバー間の通信を行うために、以下の3つのトランスポート方式が提供されています：

- **stdio** : 標準入出力を利用した通信方式。ローカル環境での開発やテストに適しています。
- **SSE (Server-Sent Events)** : サーバーからクライアントへの一方向のストリーミング通信を可能にする方式。以前はリモート通信に使用されていました。
- **Streamable HTTP** : 2025 年 3 月 26 日に公式に提唱された新しい通信方式で、SSE の後継として位置づけられています。

Streamable HTTP は、単一の HTTP エンドポイントを通じて双方向の通信を可能にし、セッション管理やストリーミング応答などの機能を提供します。これにより、リモート環境での MCP サーバーのホスティングが容易になります。

詳細は公式仕様書をご参照ください：[MCP Specification - Transports](#)

SSE から Streamable HTTP への変遷や比較がまとめられた Zenn 記事がありますので、興味のある方はぜひご覧ください：[MCP の通信方法がステートフルからステートレスに？ : HTTP+SSE から Streamable HTTP へ](#)

# Streamable HTTP 対応の MCP サーバーを構築する

前回作成した stdio ベースの MCP サーバーを、Streamable HTTP に対応させてみましょう。

## 1. 新しいサーバー ファイルの作成

まず、新しいサーバー ファイルを作成します。

```
touch streamable_http_server.ts # PowerShell: New-Item streamable_http_server.ts
```

## 2. サーバーコードの実装

以下のコードを streamable\_http\_server.ts に記述します。

```
import express from "express";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/transport/streamable-http.js";
import { z } from "zod";

const app = express();
app.use(express.json());

const transport = new StreamableHTTPServerTransport({
  sessionIdGenerator: undefined,
});

function getServer(): McpServer {
  const server = new McpServer({
    name: "example-server",
    version: "1.0.0",
  });

  server.tool(
    "add",
    "Add two numbers",
    { a: z.number(), b: z.number() },
    async ({ a, b }) => ({
      content: [{ type: "text", text: String(a + b) }],
    })
  );
}
```

```
    return server;
}

app.post("/mcp", async (req, res) => {
    // In stateless mode, create a new instance of transport and server for each request
    // to ensure complete isolation. A single instance would cause request ID collisions
    // when multiple clients connect concurrently.

    try {
        const server = getServer();
        const transport: StreamableHTTPServerTransport =
            new StreamableHTTPServerTransport({
                sessionIdGenerator: undefined,
            });
        res.on("close", () => {
            console.log("Request closed");
            transport.close();
            server.close();
        });
        await server.connect(transport);
        await transport.handleRequest(req, res, req.body);
    } catch (error) {
        console.error("Error handling MCP request:", error);
        if (!res.headersSent) {
            res.status(500).json({
                jsonrpc: "2.0",
                error: {
                    code: -32603,
                    message: "Internal server error",
                },
                id: null,
            });
        }
    }
});

// GET リクエストは SSE エンドポイントとの互換性のために実装する必要がある
// SSE エンドポイントを実装しない場合は、405 Method Not Allowed を返す
app.get("/mcp", async (req, res) => {
    console.log("Received GET MCP request");
    res.writeHead(405).end(
        JSON.stringify({
            jsonrpc: "2.0",
            error: {
                code: -32000,
                message: "Method not allowed.",
            },
        })
    );
});
```

```
        id: null,
      })
    );
  });

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

### 3. サーバーの起動

以下のコマンドでサーバーを起動します。

```
node streamable_http_server.ts
```

### 4. VS Code での設定

.vscode/settings.json に以下の設定を追加し、VS Code から MCP サーバーを利用できるようにします。

```
{
  "mcp": {
    "servers": {
      // ...Existing Servers...
      "my-streamable-http-server": {
        "url": "http://localhost:3000/mcp"
      }
    }
  }
}
```



### 動作確認

VS Code の Copilot チャットで以下のように入力して、MCP サーバーが正しく動作しているか確認してみましょう。

123 足す 456 は？



正しく動作していれば、 579 という応答が返ってくるはずです。

 ry0y4n

123 足す 456 は？

 **GitHub Copilot**

> Used 1 reference

▼ Ran **add** – my-mcp-server (MCP Server) 

**Input**

```
{  
  "a": 123,  
  "b": 456  
}
```

**Output**

```
579
```

123 足す 456 は 579 です。

## まとめ

このページでは、stdio ベースの MCP サーバーを Streamable HTTP に対応させ、リモート環境でのホスティングが可能な形にアップデートする方法を学びました。

Streamable HTTP を使用することで、クラウド環境や他のデバイスからのアクセスが容易になり、より柔軟な MCP サーバーの運用が可能になります。

## 次のステップ

次のページでは、今回開発した Streamable HTTP 対応の MCP サーバーをさらに発展させ、セッション管理機能を追加することで会話の状態を保持する方法を学びます。

# 5 - セッション管理機能をもつ MCP サーバーを開発する

## セッション管理機能をもつ MCP サーバーを開発する

このページでは、MCP サーバーにセッション管理機能を追加し、クライアントと継続的な通信が可能なステートフルなサーバーを実装します。

### このページのゴール

- セッション管理機能を持つ MCP サーバーを開発する
- Streamable HTTP を活用したステートフルな通信を体験する
- セッション管理の重要性を理解する

### ステートフルな MCP サーバーの実装

Streamable HTTP トランスポートでは、セッションの管理が可能です。  
これにより、クライアントとサーバー間の状態（context）を保持し、対話的で継続的なやりとりが実現できます。

前回の実装はステートレスで、毎回のリクエストが新しいセッションとして扱われていました。

今回は、セッション管理機能を持つステートフルな **MCP** サーバーを作成し、クライアントと継続的な通信ができるようにします。

## 1. 新しいサーバー ファイルを作成する

まずはファイルを作成しましょう。

```
touch stateful_streamable_http_server.ts # PowerShell: New-Item stateful_streamable_http_server.ts
```

以下のコードは、セッションごとに `StreamableHTTPServerTransport` を再利用し、接続状態

を維持する MCP サーバーの実装です。

```
import express from "express";
import { randomUUID } from "node:crypto";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/transport/streamablehttp.js";
import { isInitializeRequest } from "@modelcontextprotocol/sdk/types.js";
import { z } from "zod";

const app = express();
app.use(express.json());

const transports: { [sessionId: string]: StreamableHTTPServerTransport } = {};

app.post("/mcp", async (req, res) => {
  const sessionId = req.headers["mcp-session-id"] as string | undefined;
  let transport: StreamableHTTPServerTransport;

  if (sessionId && transports[sessionId]) {
    transport = transports[sessionId];
  } else if (!sessionId && isInitializeRequest(req.body)) {
    transport = new StreamableHTTPServerTransport({
      sessionIdGenerator: () => randomUUID(),
      onsessioninitialized: (sessionId) => {
        transports[sessionId] = transport;
      },
    });
  }

  transport.onclose = () => {
    if (transport.sessionId) {
      delete transports[transport.sessionId];
    }
  };

  const server = new McpServer({
    name: "example-server",
    version: "1.0.0",
  });

  server.tool(
    "add",
    "Add two numbers",
    { a: z.number(), b: z.number() },
    async ({ a, b }) => ({
      content: [{ type: "text", text: String(a + b) }],
    })
  );
});
```



```
        await server.connect(transport);
    } else {
        res.status(400).json({
            jsonrpc: "2.0",
            error: {
                code: -32000,
                message: "Bad Request: No valid session ID provided",
            },
            id: null,
        });
        return;
    }

    await transport.handleRequest(req, res, req.body);
});

const handleSessionRequest = async (
    req: express.Request,
    res: express.Response
) => {
    const sessionId = req.headers["mcp-session-id"] as string | undefined;
    if (!sessionId || !transports[sessionId]) {
        res.status(400).send("Invalid or missing session ID");
        return;
    }

    const transport = transports[sessionId];
    await transport.handleRequest(req, res);
};

app.get("/mcp", handleSessionRequest);
app.delete("/mcp", handleSessionRequest);

app.listen(3000);
```

コードを実行します。

```
node stateful_streamable_http_server.ts
```



## 2. クライアント コードでセッションを体感する

続いてクライアントを作成します。

```
touch client.ts # PowerShell: New-Item client.ts
```

このクライアントは、MCP サーバーと Streamable HTTP 経由で接続し、対話的にツールの呼び出しやセッション管理が行えます。

```
// 今回のクライアント コードは https://azukiazusa.dev/blog/mcp-server-streamable-http-client
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StreamableHTTPClientTransport } from "@modelcontextprotocol/sdk/client/transport.js";
import {
  CallToolRequest,
  CallToolResultSchema,
  ListToolsRequest,
  ListToolsResultSchema,
} from "@modelcontextprotocol/sdk/types.js";
import { createInterface } from "readline/promises";

let sessionId: string | undefined;
let transport: StreamableHTTPClientTransport | undefined;

const client = new Client({
  name: "example-stateful-client",
  version: "0.0.1",
});

const readline = createInterface({
  input: process.stdin,
  output: process.stdout,
});

async function main() {
  transport = new StreamableHTTPClientTransport(
    new URL("http://localhost:3000/mcp"),
    { sessionId }
  );
  await client.connect(transport);
  sessionId = transport.sessionId;

  console.log("Session ID:", sessionId);

  while (true) {
    console.log("available commands:");
    console.log("1. list-tools");
  }
}
```

```
    console.log("2. call-tool");
    console.log("3. exit");
    console.log("4. terminate-session");
    console.log("-----");

    const answer = await readline.question("Enter your input: ");

    switch (answer) {
      case "list-tools":
        await listTools();
        break;
      case "call-tool":
        await callTool();
        break;
      case "terminate-session":
        await terminateSession();
        break;
      case "exit":
        await disconnect();
        return;
      default:
        console.log("Unknown input:", answer);
    }
  }
}

async function disconnect() {
  await transport?.close();
  await client.close();
  readline.close();
  console.log("Disconnected.");
  process.exit(0);
}

async function listTools() {
  const req: ListToolsRequest = {
    method: "tools/list",
    params: {},
  };

  const res = await client.request(req, ListToolsResultSchema);

  if (res.tools.length === 0) {
    console.log("No tools available.");
  } else {
    res.tools.forEach((tool) => {
      console.log(`Tool: ${tool.name}`);
      console.log(`Desc: ${tool.description}`);
    });
  }
}
```

```
        console.log("-----");
    });
}

async function callTool() {
    const a = Number(await readline.question("Enter first number: "));
    const b = Number(await readline.question("Enter second number: "));

    const req: CallToolRequest = {
        method: "tools/call",
        params: {
            name: "add",
            arguments: { a, b },
        },
    };

    try {
        const res = await client.request(req, CallToolResultSchema);
        console.log(`${a} + ${b} =`);
        res.content?.forEach((item) => {
            if (item.type === "text") console.log(item.text);
        });
        console.log("-----");
    } catch (err) {
        console.error("Error:", err);
    }
}

async function terminateSession() {
    if (!transport) {
        console.log("No active session");
        return;
    }
    await transport.terminateSession();
    console.log("Session terminated.");
    sessionId = undefined;
}

main();
```

### 3. 実行して体験してみよう

```
npx tsx client.ts
```



表示例：

```
Session ID: d0abcdf6-c404-4e13-8539-941b23df3255
available commands:
1. list-tools
2. call-tool
3. exit
4. terminate-session
-----
Enter your input:
```



この状態で、`list-tools` や `call-tool` と入力すると、MCP サーバー上のツールを問題なく実行できます。これはセッションが継続しているためです。

```
rhyakuta@MOMOSUKE-Work Education-01
> sample-mcp-server > main ~1 > npx tsx client.ts
Session ID: d47f75a5-9001-45c9-b2d3-0059de03786c
available commands:
1. list-tools
2. call-tool
3. exit
4. terminate-session
-----
Enter your input: call-tool
Enter the first number: 12
Enter the second number: 234
Tool response: 12 + 234 =
246
```

次に、`terminate-session` を実行してみましょう。するとセッションが正常に終了し、`sessionId` が破棄されます。

その後、再び `list-tools` や `call-tool` を実行すると、以下のようにセッションが存在しないことを示すエラーが返されます。

```
Enter your input: terminate-session
Session terminated.
Session ID: undefined
available commands:
1. list-tools
2. call-tool
3. exit
4. terminate-session
-----
❖ Enter your input: list-tools
C:\Users\rhyakuta\Codes\sample-mcp-server\node_modules\@modelcontextprotocol\sdk\dist\cjs\client\streamableHttp.js:267
      throw new Error(`Error POSTing to endpoint (HTTP ${response.status}): ${text}`);
      ^
Error: Error POSTing to endpoint (HTTP 400): {"jsonrpc":"2.0","error":{"code":-32000,"message":"Bad Request: No valid session ID provided"},"id":null}
    at StreamableHTTPClientTransport.send (C:\Users\rhyakuta\Codes\sample-mcp-server\node_modules\@modelcontextprotocol\sdk\src\client\streamableHttp.ts:414:15)
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5)

Node.js v24.1.0
```

Error: Error POSTing to endpoint (HTTP 400): {"jsonrpc":"2.0","error":{"code":

これは、セッションが終了したため、それ以降のリクエストにはセッション ID が付与されていないためです。

本来であれば、セッションが終了した場合は新しいセッションを開始するようにクライアント側で実装することが望ましいです。

## セッション管理の重要性

セッション管理を導入することで、以下のような高度な機能が実現できます：

- ユーザーごとの文脈（context）や履歴の保持
- セッションベースの認証・権限制御
- 一時的な状態（キャッシュ、会話履歴など）の保持

特に、対話型エージェントやチャット アプリケーションなどでは、セッション管理は非常に重要な基盤技術となります。

## まとめ

このシリーズでは、MCP（Model Context Protocol）を使って独自のツールを LLM に接続できる仕組みを、段階的に学んできました。

まずは stdio を使ったローカル MCP サーバーの開発から始まり、Streamable HTTP を用いたりリモート接続対応、そして今回のステートフルなセッション管理まで、**MCP** のコア概念と実装パターンを一通り体験しました。

セッション管理を導入することで、MCP サーバーは以下のような拡張性を持つようになります：

- ユーザーごとの状態（履歴や設定）を保持できる
- 複雑な対話やマルチステップ処理をサポートできる
- 認証や個別データベース連携など、プロダクション品質の設計が可能になる

MCP を活用すれば、LLM を中心とした高度なインタラクティブ アプリケーションを、自分の手で構築できます。

 自分だけのエージェントを作る自由と責任は、あなたのコードの中にあります。

今後は、この基礎を活かして、状態の永続化（データベースとの連携）、複数ツールの連携、外部 API の統合など、さらに実践的な開発に挑戦してみてください。